

Guía de Laboratorio · Sesión 3

Detección de variantes: YARA + Sigma + copiloto de IA

Módulo: Gestión de Ciber-Recuperación · **Código:** 92-EAN
Universidad Ean · Educación Continua · Docente: Jawy Andrés Romero Pinto
Duración: 60 minutos

Idea del taller

En la Sesión 2 encontraste **qué** estaba dañado. Hoy cazas al **responsable y sus variantes**: la familia **Locked3D** (la misma de los `.locked3d`) volvió recompilada como **v3.1**. Convertimos un **informe CTI** en reglas **YARA** (archivos) y **Sigma** (logs), primero a mano y luego con el **LLM local** como redactor — validando siempre contra un banco de muestras y benignos.

Repositorio del curso: `ciberrecuperacion-ean` (mismo entorno de las Sesiones 1-2).

Mapa de la hora

Tiempo	Bloque	Qué hacemos
00–08 min	Setup	Dataset del caso + <code>yara</code> instalado (o fallback)
08–30 min	Parte 1 · YARA a mano	Del informe CTI a la regla: 4 variantes, 0 falsos positivos
30–45 min	Parte 2 · Sigma en logs	<code>vssadmin delete shadows</code> sin marcar al backup legítimo
45–57 min	Parte 3 · Copiloto + auditoría	El LLM redacta; tú validas e iteras
57–60 min	Cierre	Pirámide del dolor; el validador manda

Setup (8 min)

```
cd ciberrecuperacion-ean
git pull # trae la carpeta 03 de esta sesión
cd 03-deteccion-variantes-yara-sigma-ia
python3 scripts/generar_dataset.py
sudo apt install -y yara # sin permisos: usa scripts/buscar_iocs.py
```

☑ **Checkpoint setup:** existen `datasets/caso3/muestras/` (8 archivos) y `datasets/caso3/logs/eventos.jsonl` (15 eventos).

Parte 1 · Del informe CTI a la regla YARA (22 min)

Guía completa: `03-deteccion-variantes-yara-sigma-ia/parte-1-regla-yara.md`.

1. **Lee el informe** (`datasets/caso3/informe-cti-locked3d.md`): ¿qué indicadores sobreviven de la v3 a la v3.1? ¿Cuál es el más caro de cambiar para el actor?
2. **Escribe** `reglas/locked3d.yar` partiendo del esqueleto de la guía:

```
strings:
  $magic = { 4C 33 44 21 }      // cabecera L3D! (hex)
  $score = "L0CK3D-C0RE" ascii
  $mutex = "L3D_Mutex" ascii
  $nota = "LEEME_RESCATE.txt" ascii
condition:
  $magic at 0 or 2 of ($score, $mutex, $nota) // ¿por qué no "any of them"?
```

3. Valida contra las muestras:

```
yara -r -s reglas/locked3d.yar datasets/caso3/muestras/
# fallback sin yara: python3 scripts/buscar_iocs.py datasets/caso3/muestras
```

Esperado: **4 detecciones** (variantes a, b, c_v31 y empacada), **0 benignos**. La v3.1 cae solo por `magic + mutex`: eso es cazar la familia, no el archivo.

4. Prueba de falsos positivos: con `--min 1` (equivale a `any of them`) el runbook de TI — que solo *menciona* la nota — dispara. Con `--min 2` no. Una regla se valida contra lo malicioso y lo benigno.

☑ **Checkpoint Parte 1:** tu regla caza las 4 variantes con 0 FP y sabes justificar el `$magic at 0 or 2 of`.

Parte 2 · Sigma: el comportamiento en los logs (15 min)

Guía: `parte-2-regla-sigma.md`. Hay **5 eventos** con `vssadmin` pero solo **1** es malicioso (el agente de backup usa `list/create` a diario).

1. Escribe `reglas/vssadmin_delete_shadows.yml` (anti-recuperación, T1490):

```
detection:
  seleccion:
    Image|endswith: '\vssadmin.exe'
    CommandLine|contains|all:
      - 'delete'
      - 'shadows'
condition: seleccion
```

2. Caza en los logs:

```
python3 scripts/cazar_en_logs.py reglas/vssadmin_delete_shadows.yml
datasets/caso3/logs/eventos.jsonl
```

Esperado: **1 hit** en `FIN-PC-07` (`delete shadows /all /quiet`), cero en `SRV-BACKUP`. Luego **rompe la regla**: quita el `contains|all` y cuenta los falsos positivos.

3. Ejercicio: escribe tú solo `reglas/wevtutil_limpieza.yml` (limpieza de logs, T1070.001). Esperado: **2 hits** (`cl System` / `cl Security`) sin marcar el `wevtutil` qe legítimo.

☑ **Checkpoint Parte 2:** dos reglas Sigma con los hits exactos y el backup legítimo intacto.

Parte 3 · El copiloto escribe — tú auditas (12 min)

Guía: `parte-3-copiloto-ia-reglas.md`. El informe CTI puede ser sensible → **modelo local** (regla datos→backend del curso).

```
ollama run llama3.1:8b "Eres analista de deteccion. A partir de este informe CTI, escribe UNA regla YARA que detecte TODAS las variantes de la familia (v3 y v3.1) sin falsos positivos en documentos legitimos. Prioriza los indicadores estables. Devuelve solo la regla: $(cat datasets/caso3/informe-cti-locked3d.md)"
```

Audita el borrador con el mismo banco de la Parte 1: ¿compila?, ¿caza las 4 (incluida la empacada)?, ¿0 FP con el runbook?, ¿alucinó IOCs que el informe no menciona? Luego **itera con feedback dirigido** (dile qué falló y por qué) hasta que pase el banco. Repite el flujo para la regla Sigma.

☑ **Checkpoint Parte 3:** regla YARA + Sigma generadas por el copiloto, auditadas por ti y pasando el banco completo. **El validador manda, no el modelo.**

Cierre (3 min)

Lo que te llevas: las firmas de **un archivo** caducan; las reglas sobre lo que al actor le **duele cambiar** (cabecera, mutex, comportamiento) cazan a la **familia**. El LLM aporta velocidad; el banco de pruebas, verdad; tú, criterio.

Qué alistar para la Sesión 4

Las reglas de hoy funcionando, el entorno del kit al día (`make check`), Docker operativo y el repo actualizado (`git pull`). La Sesión 4 (bóveda **immutable** WORM con MinIO Object Lock + restic/Borg) responde: cuando la regla dispara, ¿desde dónde restauras con confianza?

Referencia rápida de comandos

```
python3 scripts/generar_dataset.py # dataset del caso Locked3D
yara -r -s reglas/locked3d.yar datasets/caso3/muestras/
python3 scripts/buscar_iocs.py datasets/caso3/muestras --min 2
python3 scripts/cazar_en_logs.py reglas/<regla>.yml datasets/caso3/logs/eventos.jsonl
ollama run llama3.1:8b "...: $(cat datasets/caso3/informe-cti-locked3d.md)"
```